

DRF Multi-Tenant Authorization Pattern

<https://gemini.google.com/app/e782b8456cb4cdc5>

User prompt: "I do not want Rule 1, no cross local access and no member acting with leadership permissions, enforced by remembering to add a check in every view. That fails the moment someone adds a new endpoint next year and forgets. Show me a pattern in Django REST Framework where every viewset inherits from a shared base class that filters by the requesting user's local_id and checks their role automatically, so a new endpoint is unsafe only if someone deliberately skips the shared base class, which should stand out clearly in code review. Then give me a test that creates a leadership user in Local A and a member in Local B, and checks that every relevant request across that boundary is rejected"

Response: Base ViewSet Pattern

This pattern enforces multi-tenant scoping (`local_id`) and leadership authorization (`role == 'leadership'`) at the viewset architecture level. Any endpoint extending `BaseLocalViewSet` or `BaseLeadershipViewSet` automatically applies both filters and permissions.

```
from rest_framework import permissions, viewsets

class IsLocalLeadership(permissions.BasePermission):
    """
    Permission check enforcing Rule 1:
    1. User must be authenticated.
    2. User must have role == 'leadership'.
    """

    def has_permission(self, request, view):
        user = request.user
        if not user or not user.is_authenticated:
            return False
        return getattr(user, "role", None) == "leadership"

class BaseLocalViewSet(viewsets.ModelViewSet):
    """
    Base viewset enforcing local_id boundary isolation across queries.
    Prevents cross-local data leaks automatically.
    """

    def get_queryset(self):
        queryset = super().get_queryset()
        user = self.request.user
        if not user or not user.is_authenticated:
            return queryset.none()
        # Strictly scope all read/write queries to the user's local_id
        return queryset.filter(local_id=user.local_id)

class BaseLeadershipViewSet(BaseLocalViewSet):
    """
    Base viewset enforcing both:
    1. local_id query scoping (via BaseLocalViewSet)
    2. Leadership role requirement
    """

    permission_classes = [IsLocalLeadership]
```

Cross-Local Boundary & Authorization Test

This Pytest suite creates a leadership user in Local A (local_id=1) and a regular member in Local B (local_id=2), attempting cross-local and unauthorized member actions across the endpoints.

```
import pytest
from rest_framework import status
from rest_framework.test import APIClient

# Replace with your actual model factories/imports
from core.models import Announcement, Local, Member

@pytest.fixture
def locals_setup(db):
    local_a = Local.objects.create(name="Local 101")
    local_b = Local.objects.create(name="Local 202")
    return local_a, local_b

@pytest.fixture
def leader_local_a(db, locals_setup):
    local_a, _ = locals_setup
    return Member.objects.create(
        full_name="Alice Leader",
        email="alice@locala.org",
        classification="Inside Wireman",
        local=local_a,
        role="leadership",
        status="active",
    )

@pytest.fixture
def member_local_b(db, locals_setup):
    _, local_b = locals_setup
    return Member.objects.create(
        full_name="Bob Member",
        email="bob@localb.org",
        classification="Apprentice",
        local=local_b,
        role="member",
        status="active",
    )

@pytest.fixture
def announcement_local_a(db, locals_setup, leader_local_a):
    local_a, _ = locals_setup
    return Announcement.objects.create(
        local=local_a,
        title="Emergency Meeting Local A",
        body="Details here",
        needs_ack=True,
    )

@pytest.mark.django_db
class TestAccessControlBoundaries:

    def test_cross_local_access_denied_for_leader(
```

```

        self, leader_local_a, member_local_b
    ):
        """
        Verify that even a Leadership user in Local A cannot read or access
        member data belonging to Local B.
        """
        client = APIClient()
        client.force_authenticate(user=leader_local_a)

        # GET detail endpoint for Local B member
        url = f"/api/v1/members/{member_local_b.id}/"
        response = client.get(url)

        # Should return 404 Not Found because get_queryset() filters out Local B entirely
        assert response.status_code == status.HTTP_404_NOT_FOUND

    def test_member_cannot_perform_leadership_action(
        self, member_local_b, announcement_local_a
    ):
        """
        Verify that a regular Member in Local B is denied access when attempting
        leadership actions or accessing endpoints restricted to Leadership.
        """
        client = APIClient()
        client.force_authenticate(user=member_local_b)

        # Attempt to access leadership announcement endpoint
        url = f"/api/v1/announcements/{announcement_local_a.id}/"
        response = client.get(url)

        # Forbidden due to permission_classes check (IsLocalLeadership)
        assert response.status_code == status.HTTP_403_FORBIDDEN

    def test_member_cannot_create_announcement(self, member_local_b):
        """
        Verify that regular members are blocked from posting announcements.
        """
        client = APIClient()
        client.force_authenticate(user=member_local_b)

        url = "/api/v1/announcements/"
        payload = {
            "title": "Unauthorized Announcement",
            "body": "Should fail",
            "needs_ack": False,
        }
        response = client.post(url, payload, format="json")

        assert response.status_code == status.HTTP_403_FORBIDDEN

```

User prompt: I am enforcing single delivery with a unique constraint on announcement_id and member_id, plus ON CONFLICT DO NOTHING upserts, done in batches by a Celery worker. Two cases to check. First, the same send request arrives twice because the client retried it. Does the second call trigger fan out again, and does it land safely on the same conflict safe path, or does it need its own separate dedup step at the request level. Second, the worker crashes partway through, say after batch 40 of 90, and a new worker picks up the same job. Does it start over from batch 1, and if it does, does that still land safely on the constraint, or do I also need a marker showing the job already finished.

Response: Case 1: Client Retries the Send Request (API Level)

If the client sends two identical POST requests (due to a timeout or client-side retry), executing `task.delay(announcement_id)` on both requests will enqueue two Celery tasks.

- **Without Request-Level Deduplication:**

- If the second POST request creates a **new** Announcement record in the database, it receives a new `announcement_id`. The worker will fan out tasks for the new ID, and because `(announcement_id_2, member_id)` is a distinct tuple from `(announcement_id_1, member_id)`, the `UNIQUE (announcement_id, member_id)` constraint **will not trigger**. Members will receive a duplicate notification.
- If the retry mechanism attaches an idempotency key or reuses the same `announcement_id`, the worker processes the second task for the same ID. The database `ON CONFLICT DO NOTHING` statement will catch the existing `(announcement_id, member_id)` rows and safely skip inserting them.

- **Recommendation:**

- You **need a request-level deduplication step** (such as an Idempotency-Key header stored in Redis or an explicit state check on the Announcement model) to prevent creating duplicate Announcement database records upon client retries.

Case 2: Celery Worker Crashes Mid-Job (Batch 40 of 90)

If the worker process crashes at batch 40 and Celery re-queues or restarts the task, the worker restarts execution.

- **Database Level:**

- Starting over from batch 1 is completely safe at the database level. Batches 1 through 40 will attempt `INSERT INTO announcement_recipients ... ON CONFLICT (announcement_id, member_id) DO NOTHING`. PostgreSQL will skip the existing rows without error, and batches 41 through 90 will insert as normal.

- **Push Delivery Level:**

- If push notification dispatches (APNs/FCM) occur in the same loop or step before/during row creation without atomic state updates, members in batches 1 through 40 could receive duplicate push notifications.
- If the job generates database recipient records first and dispatches pushes in a separate pipeline stage based on `delivery_status = 'pending'`, the database constraint protects row generation.

- **Job State Marker:**

- While `ON CONFLICT DO NOTHING` guarantees database row idempotency, adding job status markers (`status = 'processing'`, `status = 'completed'`) or skipping already-completed batch numbers is recommended to prevent wasting worker resources, database query cycles, and unnecessary locks on existing rows during retries.

User prompt: Separate from making these rules true, how would I find out in production if either rule had already been broken, after the fact, without reading code. For Rule 1, what is one query or log signal that would catch a cross local access if the permission layer were ever bypassed. For Rule 2, what is one query that would catch a member with more than one recipient row for the same announcement, and how often would I realistically run it.

Response: Rule 1: Audit Signal for Cross-Local Data Leakage

To detect whether a user accessed data outside their assigned `local_id`, you rely on structural database constraints where every record links to a `local_id` (either directly or via parent relations).

Production Audit Query (Application Log or APM Data)

If you log `request.user.local_id`, `request.user.id`, and the targeted resource `local_id` on every API request or database fetch, you can query for mismatches:

```
-- Query against HTTP request audit logs or structured database access logs
SELECT
    request_timestamp,
    user_id,
    user_local_id,
    accessed_resource_type,
    accessed_resource_id,
    accessed_resource_local_id
FROM application_audit_logs
WHERE user_local_id != accessed_resource_local_id
      AND user_role != 'superadmin';
```

Database Integrity Check Query

If direct cross-local relationships were mistakenly written to the database (e.g., an `announcement_recipients` record connecting a member of Local B to an announcement belonging to Local A):

```
-- Detects recipient rows where the member's local_id does not match the announcement's local_id
SELECT
    ar.id AS recipient_id,
    ar.announcement_id,
    a.local_id AS announcement_local_id,
    ar.member_id,
    m.local_id AS member_local_id
FROM announcement_recipients ar
JOIN announcements a ON ar.announcement_id = a.id
JOIN members m ON ar.member_id = m.id
WHERE a.local_id != m.local_id;
```

Rule 2: Detecting Duplicate Recipient Rows & Execution Frequency

Production Audit Query

To check if any member has duplicate recipient records for the same announcement:

```
SELECT
    announcement_id,
    member_id,
    COUNT(*) AS record_count
FROM announcement_recipients
GROUP BY announcement_id, member_id
HAVING COUNT(*) > 1;
```

Realistic Execution & Analysis

- **Result Expectation:** If your database schema includes `CONSTRAINT uq_announcement_member UNIQUE (announcement_id, member_id)`, this query will **always return 0 rows**, as PostgreSQL enforces this constraint at the engine level and blocks duplicate writes.
- **Execution Frequency:**
 - If the database constraint is **actively enforced**: You do not need to run this query on a recurring schedule. Instead, monitor **PostgreSQL error logs** for `UniqueViolation (SQLSTATE 23505)` exceptions thrown during Celery batch writes.
 - If the constraint **was temporarily omitted or disabled**: Run this query **once daily** off-peak or **immediately following any Celery worker incident/crash** during a large broadcast.

